BRANCH COVERAGE REPORT

Real-Time Chat Application Using Socket Programming

Mini Project Part-3, item 2(b)(iv)

Item	Details
Project	Real-Time Chat Application Using Socket Programming
Course	Software Engineering (UE23CS341A), PES University. Mini Project Part-3: Testing
Team	Kabir Raju G (Kabir7code) Keerthana K (KeerthanaK0304) Joshua (jjosh2005)
Repository	github.com/Kabir7code/Chat-application-Socket-Programming-
Test run	11 Nov 2026
Code tested	The complete Sprint 3 code: main on 11 Nov 2026 plus the Sprint 3 pull requests for #53, #54, #59, #60 and #61, which were being merged that day.

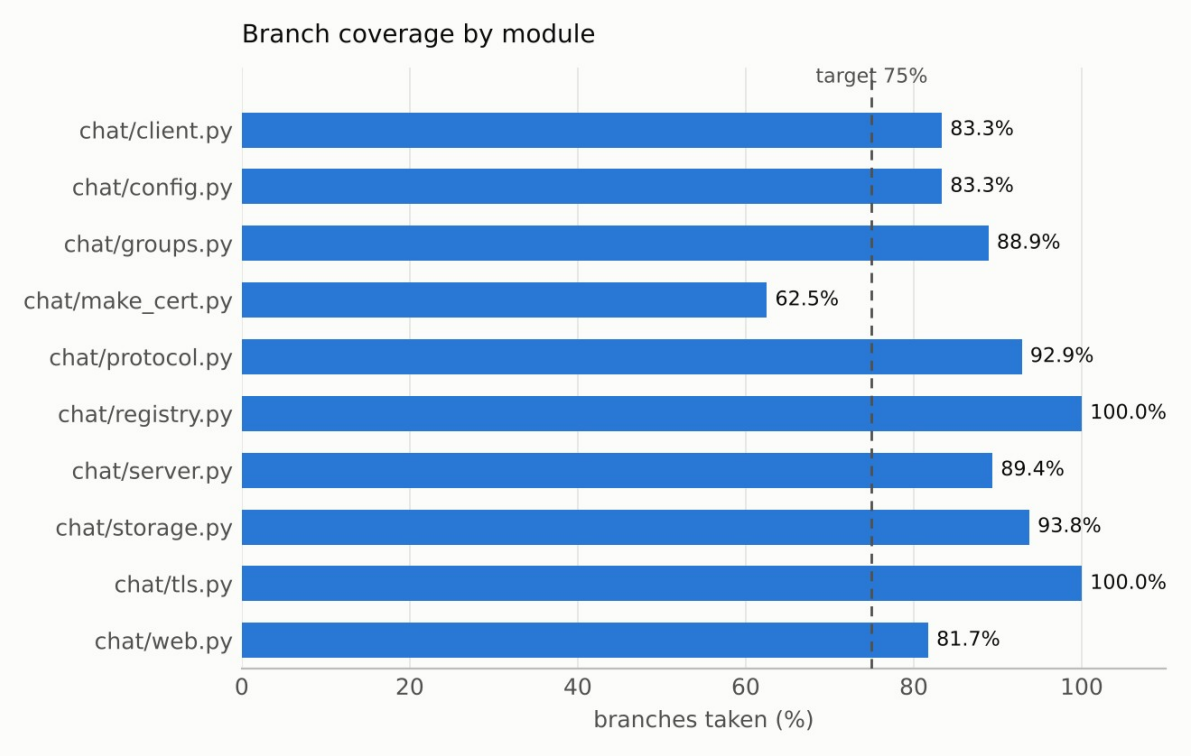
1. What was measured

Branch coverage counts, for every decision in the code (if, elif, while, for, try), whether the automated tests took each possible way out of it. A line can be run while one of its branches never is; for example an if whose condition is always true in the tests. Branch coverage therefore finds untested cases that line coverage misses.

- Tool: coverage.py 7.17 through pytest-cov, with branch measurement on (--cov-branch).
- Code measured: every module in chat/ (the tests themselves are not counted).
- Tests: the full automated suite, 338 tests, all passing. Programs started by the tests (the terminal client and the server run from the command line) are measured too (coverage subprocess patch).
- Run: 11 Nov 2026, Python 3.13. Code: The complete Sprint 3 code: main on 11 Nov 2026 plus the Sprint 3 pull requests for #53, #54, #59, #60 and #61, which were being merged that day.
- Target from the Test Plan: at least 80% of lines and 75% of branches.

2. Overall result

Measure	Result
Branches (possible ways out of a decision)	330
Branches taken by the tests	284
Branches never taken	46
Decisions only partly covered	40
Branch coverage	86.1% (target 75%: met)
Line coverage (for comparison)	91.5%



3. Coverage by module

Module	Role	Branches	Taken	Missed	Partial	Coverage
chat/client.py	terminal client	60	50	10	8	83.3%
chat/config.py	settings and command-line options	12	10	2	2	83.3%
chat/groups.py	group chats	36	32	4	4	88.9%
chat/make_cert.py	certificate tool (run by hand)	8	5	3	3	62.5%
chat/protocol.py	message framing	14	13	1	1	92.9%
chat/registry.py	connected users	6	6	0	0	100.0%
chat/server.py	chat server	94	84	10	8	89.4%
chat/storage.py	SQLite message store	16	15	1	1	93.8%
chat/tls.py	TLS encryption	2	2	0	0	100.0%
chat/web.py	web UI gateway	82	67	15	13	81.7%
Total		330	284	46	40	86.1%

4. What is not covered, and why

Module	Branches never taken (line -> next line)	What they are
chat/client.py	83->85, 142->143, 144->149, 158->159, 158->161, 174->175, 177->193, 179->180, 182->183, 186->177	code that only runs in the interactive program: the background thread that prints incoming messages, the certificate and connection error exits of main(), Ctrl+C and 'connection lost' handling, and the end-of-input and retry paths of register(). The scripted test cases run these (UT_01, ST_02, IT_26, ST_17, ST_18), but they are not part of the coverage run.
chat/config.py	102->106, 103->102	the server's TLS setup from certificate files, used only when the server is started with real certificate files.

Module	Branches never taken (line -> next line)	What they are
chat/groups.py	49->50, 107->108, 121->122, 155->exit	three error checks no automated test triggers (a saved group with no members, adding someone who is already a member, leaving a group you are not in) and recording a message for a group that was just deleted.
chat/make_cert.py	29->23, 36->37, 63->64	the command-line part of the certificate tool (python -m chat.make_cert) and its 'openssl not found' error; the team runs it once by hand.
chat/protocol.py	78->79	the case where the connection closes between a frame's length header and its body.
chat/server.py	60->55, 111->112, 111->116, 120->122, 128->exit, 273->274, 297->298, 334->exit, 432->434, 438->439	serve_forever() and the command-line start (outside the coverage run), a TCP keepalive option the operating system may not offer, stop() before start(), an admin adding a member who is offline, starting without --keep, and two error returns in private messaging.
chat/storage.py	195->193	loading a saved member row whose group no longer exists (it is skipped); the database never holds one, because members are deleted with their group.
chat/web.py	100->103, 158->159, 185->186, 189->192, 209->211, 222->exit, 281->282, 327->329, 357->358, 408->409, 412->413, 437->438, 448->449, 448->451, 465->466	main() (starting the gateway from the command line), HTTP error replies for malformed requests (missing Content-Length, a body that is not a JSON object, an unknown URL, a bad command line) and connection errors to the chat server (certificate rejected, server closed or not answering).

Most of what is missed is start-up code run from the command line, the certificate tool, and error branches for situations that are hard to cause on purpose (a database failure in one particular operation, a malformed HTTP request). The scripted test cases cover the start-up paths (ST_02, ST_19, IT_26) and a failing database (ST_22). A few user-facing error checks (leaving a group you are not in, adding an existing member, malformed web requests) have no automated test yet; adding them is the easiest way to raise coverage further.

5. How to reproduce

```
pip install -r requirements-dev.txt pytest-cov
pytest --cov=chat --cov-branch --cov-report=term-missing --cov-report=html
```

Attached in Documents/Test_Reports: coverage_html/index.html (open in a browser; every module with its missed lines in red and partly-taken branches in yellow), coverage.xml and coverage.json.